

1. Basic List Ops

Member

```
member(X,[X|_]).
member(X,[_|T]):-member(X,T).
```

Count Member (mymember/3)

```
mymember(_,[],0):-fail.
mymember(X,[X|T],N):-N1 is N-1,mymember(X,T,N1).
mymember(X,[_|T],N):-mymember(X,T,N).
```

Append

```
conc([],L,L).
conc([H|T],L,[H|R]):-conc(T,L,R).
```

Reverse

```
rev(L,R):-rev_acc(L,[],R).
rev_acc([],A,A).
rev_acc([H|T],A,R):-rev_acc(T,[H|A],R).
```

Length

```
len([],0).
len([_|T],N):-len(T,M),N is M+1.
```

Nth Element

```
nth(1,[H|_],H).
nth(N,[_|T],X):-N>1,N1 is N-1,nth(N1,T,X).
```

Last

```
last([X],X).
last([_|T],X):-last(T,X).
```

2. Deletion / Search

Delete First Occurrence

```
mydelete(X,[X|T],T):-!.
mydelete(X,[H|T],[H|R]):-mydelete(X,T,R).
```

Delete All

```
del_all(_,[],[]).
del_all(X,[X|T],R):-del_all(X,T,R).
del_all(X,[H|T],[H|R]):-X\=H,del_all(X,T,R).
```

Select

```
select(X,[X|T],T).
select(X,[H|T],[H|R]):-select(X,T,R).
```

Next To (Adjacency)

```
mynextto([A,B|_],A,B).
mynextto([_|T],A,B):-mynextto(T,A,B).
```

3. Combinatorics

Permutation

```
perm([],[]).
perm(L,[H|T]):-select(H,L,R),perm(R,T).
```

Subset

```
subset([],[]).
subset([H|T],[H|R]):-subset(T,R).
subset([_|T],R):-subset(T,R).
```

Sublist

```
sublist(S,L):-conc(_,X,L),conc(S,_,X).
```

Remove Duplicates (Set)

```
removedups([],[]).
removedups([H|T],R):-member(H,T),!,removedups(T,R).
removedups([H|T],[H|R]):-removedups(T,R).
```

4. Partitioning

List Partition (No Dups)

```
mylistpartition([],[],[]).
mylistpartition([H|T],[H|B],C):-mylistpartition(T,B,C).
```

```
mylistpartition([H|T],B,[H|C]):-mylistpartition(T,B,C).
Set Partition (Handles Dups)
mysetpartition(A,B,C):-removedups(A,S),mylistpartition(S,B,C).
Intersection / Union
inter([],_,[]).
inter([H|T],L,[H|R]):-member(H,L),!,inter(T,L,R).
inter([_|T],L,R):-inter(T,L,R).
union([],L,L).
union([H|T],L,R):-member(H,L),!,union(T,L,R).
union([H|T],L,[H|R]):-union(T,L,R).
```

5. Trees & Relations

Descendant

```
mydescendant(X,Y):-parent(Y,X).
mydescendant(X,Y):-parent(Y,Z),mydescendant(X,Z).
```

6. Peano Arithmetic

Addition

```
add(X,0,X).
add(X,s(Y),s(Z)):-add(X,Y,Z).
```

Multiplication

```
mult(_,0,0).
mult(X,s(Y),Z):-mult(X,Y,P),add(P,X,Z).
```

7. Numbers as Lists

List → Number

```
listnum(L,N):-listnum(L,0,N).
listnum([],A,A).
listnum([H|T],A,N):-A1 is A*10+H,listnum(T,A1,N).
```

Higher Permutation Test (HP)

```
myhp(N1,N2):-perm(N2,N1),listnum(N1,X),listnum(N2,Y),X>Y.
```

Next Higher Permutation

```
mynhp(N1,N2):-myhp(N1,N2),
\+ (myhp(N,N2),listnum(N,A),listnum(N1,B),A<B).
```

8. Logic & Control

- Cut (!)
- Green Cut: prunes search, no semantic change.
 - Red Cut: changes meaning (danger).
 - Pattern: rule :- cond, !, body.

Negation as Failure (\+)

```
not(G):-G,!,fail.
not(_).
```

IMPORTANT PROPERTY:

- \+ G is sound ONLY if G is ground.
- If G has uninstantiated variables, negation becomes unsound.

Disjoint Sets

```
disjoint(A,B):-member(X,A),member(X,B),!,fail.
disjoint(_,_).
```

9. CNF Evaluation

Literal Eval

```
val(a,[A|_],A). val(b,[_|[B|_]],B).
val(c,_,C). val(d,_,D). val(e,_,E). % compress manually
ev(not(X),I):-val(X,I,f).
ev(X,I):-val(X,I,t).
```

Clause / CNF

```
cl([L|_],I):-ev(L,I),!.
cl([_|T],I):-cl(T,I).
```

```
cnf([],_).
cnf([C|T],I):-cl(C,I),cnf(T,I).
```

```
mysatisfied(F):-interpretation(I),cnf(F,I),!.
```